

Sujet de stage

Conception et développement d'une plateforme collaborative de gestion des tâches basée sur une architecture microservices et une communication événementielle avec Apache Kafka

Contexte

Dans un environnement professionnel, les équipes travaillent simultanément sur plusieurs projets impliquant différents collaborateurs, rôles et responsabilités. Les outils de gestion de tâches existants sont parfois complexes, peu adaptés aux besoins spécifiques des entreprises ou difficiles à personnaliser.

L'objectif de ce projet est de concevoir et développer une plateforme web moderne permettant aux équipes de collaborer efficacement sur leurs projets grâce à une architecture microservices offrant modularité, évolutivité et facilité de maintenance.

Le projet mettra l'accent sur les bonnes pratiques de développement backend avec **NestJS**, la communication asynchrone entre microservices via **Apache Kafka**, la mise en cache avec **Redis**, le développement d'une interface utilisateur moderne avec **Angular**, ainsi que l'automatisation des processus d'intégration et de déploiement grâce à **GitLab CI/CD**.

Objectifs du projet

Le projet a pour objectif de développer une application collaborative permettant de :

- gérer plusieurs organisations ou équipes ;
- créer et administrer des projets ;
- planifier et organiser les tâches ;
- attribuer des tâches aux collaborateurs ;
- suivre l'avancement des projets en temps réel ;
- envoyer automatiquement des notifications lors des différents événements ;
- garantir une architecture évolutive basée sur des microservices indépendants ;
- assurer une communication distribuée basée sur une architecture orientée événements (**Event-Driven Architecture**).

Architecture générale

L'application sera construite selon une architecture **Microservices** afin de séparer les responsabilités métier en plusieurs services indépendants.

Chaque microservice possédera sa propre logique métier et sa propre base de données afin de garantir l'indépendance des services.

Les échanges entre microservices seront réalisés de manière asynchrone grâce à **Apache Kafka** selon une approche événementielle.

Les services publieront des événements métiers dans des **topics Kafka**, tandis que les services intéressés consommeront ces événements afin d'exécuter leurs traitements respectifs.

Une **API Gateway** servira de point d'entrée unique entre le frontend Angular et les différents microservices.

L'ensemble des services sera conteneurisé avec Docker afin de faciliter le développement, les tests et le déploiement.

Microservices

1. Auth Service

Responsable de :

- l'authentification des utilisateurs ;
- la gestion des comptes ;
- la connexion sécurisée ;
- la génération des JWT ;
- la gestion des Refresh Tokens ;
- la gestion des rôles et permissions.

2. User Service

Permet de gérer :

- les utilisateurs ;
- les profils ;

- les équipes ;
- les rôles des collaborateurs ;
- les informations personnelles.

3. Project Service

Responsable de :

- la création des projets ;
- la gestion des membres ;
- les catégories de projets ;
- les échéances ;
- les tableaux Kanban.

4. Task Service

Permet de gérer :

- les tâches ;
- les sous-tâches ;
- les priorités ;
- les statuts ;
- les échéances ;
- l'attribution aux utilisateurs ;
- les commentaires ;
- l'historique des modifications.

5. Notification Service

Responsable de :

- l'envoi des notifications ;
- les rappels des échéances ;
- les notifications lors des changements de statut ;
- les notifications lors de l'attribution d'une tâche.

Ce service consommera les événements publiés dans Kafka.

Communication entre microservices avec Apache Kafka

Les échanges entre services seront réalisés grâce à **Apache Kafka**.

Kafka jouera le rôle de plateforme centrale de diffusion des événements.

Chaque service pourra :

- publier des événements (**Kafka Producer**) ;
- consommer des événements (**Kafka Consumer**) ;
- traiter uniquement les événements nécessaires.

Exemple d'un flux événementiel

Création d'une tâche

1. L'utilisateur crée une tâche depuis Angular.
2. Le Task Service enregistre la tâche :

Task PostgreSQL

3. Le Task Service publie un événement Kafka :

Topic :

task-events

Message :

```
{  
  "event": "TaskCreated",  
  "taskId": 10,  
  "assignedUser": 5  
}
```

4. Le Notification Service consomme l'événement.
5. Une notification est créée :

Nouvelle tâche assignée

Mise en cache

Redis sera utilisé pour :

- mettre en cache les tableaux de bord ;
- accélérer les recherches fréquentes ;
- stocker temporairement certaines données ;
- limiter les accès répétés aux bases PostgreSQL ;
- améliorer les performances des API.

Base de données

Chaque microservice disposera de sa propre base PostgreSQL.

Exemple :

Auth Database

User Database

Project Database

Task Database

Notification Database

Cette séparation respecte le principe :

Database per Service

et garantit l'indépendance des microservices.

Frontend Angular

L'application Angular offrira une interface moderne permettant de :

- gérer les utilisateurs ;
- gérer les projets ;
- créer et suivre les tâches ;
- consulter les tableaux de bord ;
- recevoir les notifications en temps réel.

Technologies utilisées

Backend

- NestJS
- TypeScript
- PostgreSQL
- Apache Kafka
- Redis
- JWT
- Swagger

Frontend

- Angular
- Angular Material
- RxJS

DevOps

- Docker
- Docker Compose
- GitLab CI/CD
- Railway
- Vercel